

House Price Prediction via Tree-Based Models, Coordinate Encoding, and Price-per-sqft Stacking

Seungho Han 20211356

Moonseong Son 20221222

Sungho Kim 20241065

Jaeyun Lim 20251303

Team 8 · June 9, 2026

Code: <https://github.com/pw4ch7re3/house-price-regressor>

1 Introduction

House price prediction is a supervised regression task: given property attributes, estimate a continuous sale price. The *USA House Prices* dataset used here has three characteristics that shape our approach. First, several of the most informative features are not natively numeric: location fields such as `city` and `zipcode` carry strong price signal yet cannot be dropped or naively cast to integers. Second, the target price is severely right-skewed with a few extreme outliers, so we remove only the most extreme cases and retain realistic noise. Third, the feature space is imbalanced—living area (`sqft_living`) dominates a tree’s feature importance, so a model can score reasonably while essentially predicting from size alone.

This motivates our central hypothesis: a model that over-relies on the dominant size feature leaves predictive signal on the table, and richer representations—through more expressive model structure or better feature encoding—should recover it. We test this with four models spanning that spectrum: a Multi-Layer Perceptron (MLP), a Decision Tree (DT), a Random Forest (RF), and a Multi-Layered Gradient Boosting Decision Tree (mGBDT) [1]. We further study two domain-aware ideas: *coordinate encoding*, which converts address fields into geographic coordinates, and *price-per-sqft stacking*, which blends a direct price model with a price-per-square-foot model to dampen the influence of size and of price outliers. Throughout, the naive baseline is a Random Forest on ordinal-encoded categories (`rf/cat`), and we ask which combination of model, encoding, and stacking best improves over it.

2 Background and Related Work

2.1 Decision and Regression Trees

A Decision Tree recursively partitions the feature space using binary questions on single features (e.g., `sqft_living` $\leq$ 2915), storing a prediction at each leaf [3]. Trees handle mixed feature types, need no scaling, and are interpretable, but a single tree is unstable and overfits easily—motivating the ensembles below. For a continuous target the tree becomes a **regression tree**: rather than fitting one global formula (as linear regression does), it partitions the space into local regions and predicts the sample mean in each [5]. Letting S be the samples reaching a node, the chosen split maximises the reduction in sum of squared errors and each leaf S_ℓ predicts the mean:

$$\Delta\text{SSE} = \text{SSE}(S) - \text{SSE}(S_L) - \text{SSE}(S_R), \quad \text{SSE}(S) = \sum_{i \in S} (y_i - \bar{y}_S)^2, \quad \hat{y}_\ell = \frac{1}{|S_\ell|} \sum_{i \in S_\ell} y_i. \quad (1)$$

Because features such as `sqft_living`, `waterfront`, and `view` interact non-linearly, a regression tree is a natural baseline here.

2.2 Gradient Boosting and Random Forests

Gradient Boosting [2] combines many *shallow* trees additively, building the ensemble stage by stage so that each new tree corrects the current residual:

$$F_K(x) = F_0(x) + \eta \sum_{k=1}^K h_k(x), \quad r_i^{(k)} = y_i - F_{k-1}(x_i), \quad (2)$$

where F_0 is an initial prediction, η the learning rate, and h_k fits the negative gradient (the residual for squared error). A common refinement, *stochastic gradient boosting*, fits each tree on a random subsample to reduce variance [6]; the XGBoost modules used in our mGBDT support this.

A **Random Forest** [3] instead reduces variance by *bagging*: it trains many trees on bootstrap resamples and considers a random feature subset at each split, so averaging the de-correlated trees cancels per-tree noise. Being robust and low-tuning, a Random Forest on the naively (ordinal) encoded categories serves as our baseline.

2.3 Multi-Layered Gradient Boosting Decision Trees

Standard GBDTs learn a *flat* ensemble—all trees share one input representation—whereas deep networks learn *hierarchical* representations through stacked layers. The mGBDT framework [1] bridges this gap by stacking M GBDT modules, the output of layer l feeding layer $l + 1$ ($o_0 = x$, $o_l = F_l(o_{l-1})$). Since GBDTs are non-differentiable, mGBDT replaces back-propagation with *target propagation*: the final layer is trained toward y , and inverse mappings G_l propagate pseudo-labels backward ($\tilde{o}_{l-1} = G_l(\tilde{o}_l)$, $\tilde{o}_M = y$) so each layer F_l receives its own target. The project hypothesis is that this layered representation captures hierarchical feature interactions better than a flat GBDT.

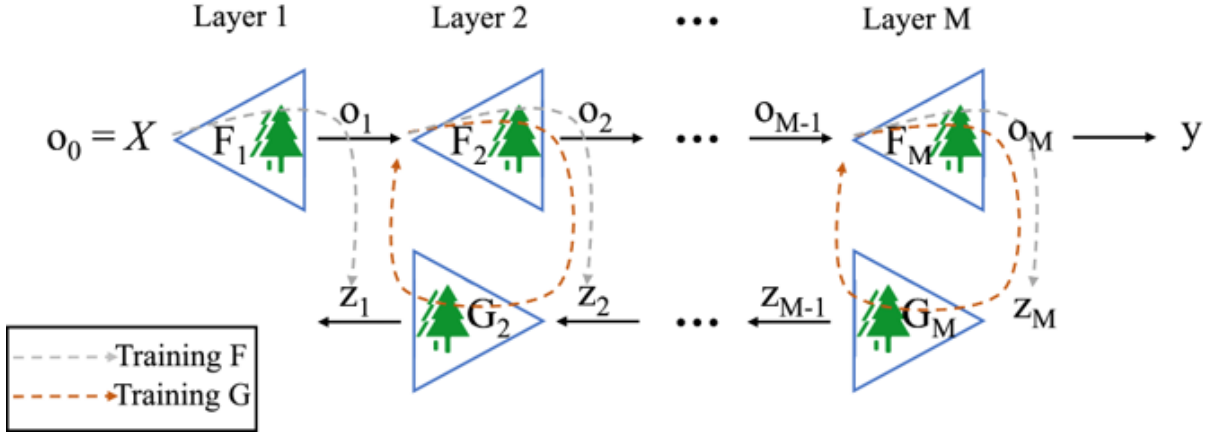


Figure 1: mGBDT architecture

We instantiate F_l and G_l with XGBoost [4] modules. A two-layer mGBDT was tried, but its second layer learned noise and *degraded* accuracy (Section 5); the single-layer configuration we report—one target-propagation layer over XGBoost—reduces to standard gradient boosting, so our “mGBDT” results are effectively those of plain XGBoost, which we adopt as the final boosting model.

3 Dataset

3.1 Source and Overview

We use the *USA House Prices* dataset published on Kaggle ¹, which contains 4,140 residential property sales records from Washington State. The raw dataset comprises 18 columns covering structural attributes, location information, and transaction metadata. Preprocessing removes records with a non-positive price and the most extreme price outliers (the top 0.1% by price), leaving **4,086** records; this caps only a handful of pathological sales while deliberately retaining realistic noise. We then remove non-informative fields (**date**, **street**, **statezip**, **country**) and apply the feature engineering described in Section 4, yielding **17** model-ready input features. All reported numbers

¹<https://www.kaggle.com/datasets/fratzcan/usa-house-prices/data>

come from 5-fold cross-validation over these 4,086 records (about 3,269 training / 817 validation rows per fold); we do not use a single fixed train/test split.

3.2 Prediction Target

The single reported prediction target is **price**, the raw transaction price in USD. As is typical for residential price prediction, raw price is strongly correlated with living area (**sqft_living**): larger homes tend to be more expensive simply by virtue of their size, which can cause a model to over-rely on size-related features.

To counteract this, we additionally derive an auxiliary quantity, **price_per_sqft**:

$$\text{price_per_sqft} = \frac{\text{price}}{\text{sqft_living}}. \quad (3)$$

Normalising sale price by interior area—analogueous to the “price per square metre” convention in real estate valuation—decouples the size effect from the price signal and tames price outliers (an extreme *price* need not be an extreme *price-per-sqft*). Crucially, **price_per_sqft** is *not* reported as a separate regression task. Instead it is used inside the stacking ensemble of Section 4: a price-per-sqft model is trained alongside a direct price model, its prediction is converted back to a price via $\times \text{sqft_living}$, and the two are blended. All four models (**MLP**, **DT**, **RF**, **mGBDT**) are trained and evaluated on the **price** target, both as single models and as this stacked ensemble.

3.3 Feature Description

Table 1 lists all 17 features used as model inputs after preprocessing. Removed raw columns (**sqft_lot**, **yr_built**, **yr_renovated**, **lat**, **long**) are replaced by their engineered counterparts. The location fields **city** and **zipcode** are encoded differently per experimental variant (ordinal codes vs. target encoding), and the coordinate features **x**, **y**, **z** are present only in the coordinate-aware variants; see Section 4.

Table 1: Feature descriptions after preprocessing.

Feature	Type	Description
bedrooms	Numeric	Number of bedrooms
bathrooms	Numeric	Number of bathrooms (0.5-unit increments)
sqft_living	Numeric	Interior living area (sq. ft.)
floors	Numeric	Number of floors
waterfront	Binary	Waterfront property (0 = No, 1 = Yes)
view	Ordinal	View quality score (0–4)
condition	Ordinal	Property condition score (1–5)
sqft_above	Numeric	Above-ground floor area (sq. ft.)
sqft_basement	Numeric	Basement area (sq. ft.)
city	Categorical (encoded)	City name; ordinal code or target-encoded per variant
x	Numeric	3-D Cartesian coordinate: $\cos(\text{lat}) \cdot \cos(\text{lon})$
y	Numeric	3-D Cartesian coordinate: $\cos(\text{lat}) \cdot \sin(\text{lon})$
z	Numeric	3-D Cartesian coordinate: $\sin(\text{lat})$
zipcode	Categorical (encoded)	ZIP code; ordinal code or target-encoded per variant
log_sqft_lot	Numeric	$\log(1 + \text{sqft_lot})$; log-transform of lot area to reduce skew
was_renovated	Binary	Renovation history (0 = Never, 1 = Renovated)
age	Numeric	Building age at time of sale: sale year – year built

4 Method

4.1 Research Goal

This study compares four regression models—MLP, Decision Tree (DT), Random Forest (RF), and mGBDT—on the **price** target, and asks whether richer feature encoding or model structure improves over a naive baseline (a Random Forest on ordinal-encoded categories). Two axes are crossed under identical preprocessing and evaluation: (i) the *location-encoding variant* (**cat**, **tgt**, **coord_only**, **tgt_only**), and (ii) whether each model is trained as a single price predictor or as a price / price-per-sqft *stacking ensemble*. This yields 4 models $\times$ 4 variants $\times$ 2 modes = 32 configurations, each evaluated by 5-fold cross-validation.

4.2 Preprocessing

Raw features are transformed as follows; all fold-dependent steps are fit on the training rows only.

Outliers and continuous features. Records with non-positive price and the top 0.1% of prices are dropped, leaving 4,086 records. **sqft_lot** is replaced by **log_sqft_lot** = $\log(1 + \text{sqft_lot})$ to reduce skew, and the coordinates **lat/long** are projected onto the unit sphere (*coordinate encoding*):

$$x = \cos(\text{lat}) \cos(\text{lon}), \quad y = \cos(\text{lat}) \sin(\text{lon}), \quad z = \sin(\text{lat}), \quad (4)$$

which avoids the ± 180 meridian discontinuity; missing coordinates use the per-city (else global) mean.

Temporal and scaling. We derive **age** = **sale_year** − **yr_built** and a binary **was_renovated**, then drop the raw year columns. For scaling, **x**, **y**, **z**, **condition**, **age**, **bedrooms**, **bathrooms**, **floors**, **view** are MinMax-scaled to $[0, 1]$, while **sqft_living**, **sqft_above**, **sqft_basement**, **log_sqft_lot** and the encoded **city**, **zipcode** are Z-scored.

Categorical and target. **city** and **zipcode** are encoded either as ordinal codes (naive baseline) or via leakage-free *target encoding*—each category replaced by a smoothed mean of training price (smoothing 10). **street**, **statezip**, **country**, **date** are removed as redundant. The **price** target is MinMax-scaled for training and inverted to USD before scoring (no log-transform), so all errors stay interpretable in dollars; the auxiliary **price_per_sqft** = **price**/**sqft_living** is used only by the stacking ensemble (Section 4.5).

4.3 Location-Encoding Variants

We isolate the contribution of each location signal with four feature variants, all sharing the same split and scaling:

- **cat** — ordinal codes for **city/zipcode**, no coordinates (the naive baseline encoding).
- **tgt** — target-encoded **city/zipcode** *plus* the cartesian coordinates **x,y,z**.
- **coord_only** — coordinates **x,y,z** only (**tgt** with **city/zipcode** dropped).
- **tgt_only** — target-encoded address only (**tgt** with **x,y,z** dropped).

4.4 Model Configuration

All four models use the fixed configurations in Table 2, which produce every reported number. (Exploratory tuning scripts exist under **experiments/**, but the comparison holds configurations fixed so that differences are attributable to encoding and stacking, not per-cell tuning.) The **DT** is a single CART regressor [3] (interpretable lower bound); the **RF** is a bagged ensemble of 100 unpruned trees, and RF/**cat** is the headline baseline; the **MLP** is a [32, 32] ReLU network trained with Adam. The **mGBDT** uses XGBoost forward/inverse modules [4] under target propagation; the reported *single-layer* configuration reduces to standard XGBoost (a two-layer variant performed worse, Section 5), so it is labelled “mGBDT (XGBoost)”.

Table 2: Fixed model configurations used for all reported runs.

Model	Configuration		
DT	max_depth= 6	min_samples_split= 2	min_samples_leaf= 2
RF	n_estimators= 100	max_depth=None	min_samples_leaf= 2
MLP	hidden [32, 32], Adam	lr = 10 ⁻² , batch 16	≤ 50 epochs, patience 5
mGBDT	1 TP layer over XGBoost	lr = 0.1, max_depth= 3	5 boost rounds, 20 epochs

4.5 Price / Price-per-sqft Stacking Ensemble

To dampen the dominance of size and the influence of price outliers, each model can also be trained as a two-member stacking ensemble. On every fold we train one model to predict **price** and a second, identical model to predict **price_per_sqft**; the second model’s prediction is converted back to a price estimate $\hat{p}_2 = \widehat{\text{pps}} \times \text{sqft_living}$. The final prediction blends the two,

$$\hat{p} = w \hat{p}_1 + (1 - w) \hat{p}_2, \quad (5)$$

where the weight $w \in [0, 1]$ is grid-searched (101 steps) to minimize RMSE on that fold’s *training* split before being applied to its validation split.

4.6 Experimental Protocol

All results use 5-fold cross-validation (shuffled, seed 42) over the 4,086 cleaned records, with fold indices shared across every model, variant, and mode for fairness. Target encoding, scaling, and the ensemble blend weight are fit on training rows only, so no validation information leaks. For each configuration we report the across-fold *mean* $\pm$ *std* of every metric on both splits; the single **price** target is used throughout.

4.7 Evaluation Metrics

We report five complementary regression metrics. RMSE squares residuals and so penalises large mis-pricings; MAE weights all residuals equally and is outlier-robust (a large RMSE–MAE gap flags outlier dominance); MAPE is a scale-free percentage error; R^2 is the explained variance fraction; and Adjusted R^2 corrects R^2 for the number of predictors k for fair comparison across models:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}, \quad \text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad \text{MAPE} = \frac{1}{n} \sum_i \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\%, \quad (6)$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad \bar{R}^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - k - 1}. \quad (7)$$

We rank configurations primarily by **Adjusted R^2 on the validation split**. Secondary efficiency indicators (training time, inference time, peak memory) are not recorded in the current pipeline and are left to future work.

5 Experiments

All numbers are 5-fold cross-validation means on the validation (“test”) split of the **price** target (USD). We first compare every model across the four location-encoding variants (Tables 3–4), then report full metrics for the headline configurations (Table 5). The naive baseline throughout is **rf/cat**.

5.1 Model $\times$ Encoding Comparison

Two trends are immediate. First, moving from the naive **cat** encoding to any coordinate- or target-aware variant lifts every model substantially: the baseline RF rises from 0.667 (**cat**) to 0.754 (**tgt**), and the worst cell, DT/**cat** at 0.443, is far below any encoded configuration. Figure 2 shows this effect for RF in isolation. Second, stacking the price and price-per-sqft models adds a further gain for the stronger models—most notably RF, whose best ensemble (**coord_only**, 0.794) exceeds its best single model (0.754) and is the top configuration overall.

Table 3: Single models — validation Adjusted R^2 (mean $\pm$ std) by encoding variant.

Model	cat	coord_only	tgt_only	tgt
MLP	0.533 ± 0.082	0.680 ± 0.069	0.717 ± 0.053	0.724 ± 0.045
DT	0.443 ± 0.085	0.608 ± 0.054	0.645 ± 0.061	0.633 ± 0.072
RF	0.667 ± 0.047	0.753 ± 0.046	0.743 ± 0.048	0.754 ± 0.048
mGBDT (XGBoost)	0.665 ± 0.044	0.744 ± 0.045	0.753 ± 0.041	0.765 ± 0.043

Table 4: Stacking ensembles — validation Adjusted R^2 (mean $\pm$ std) by encoding variant.

Model	cat	coord_only	tgt_only	tgt
MLP	0.581 ± 0.047	0.698 ± 0.044	0.749 ± 0.047	0.779 ± 0.050
DT	0.598 ± 0.035	0.718 ± 0.050	0.716 ± 0.054	0.713 ± 0.053
RF	0.739 ± 0.053	0.794 ± 0.048	0.770 ± 0.053	0.782 ± 0.053
mGBDT (XGBoost)	0.698 ± 0.052	0.753 ± 0.049	0.762 ± 0.058	0.771 ± 0.051

5.2 Headline Configurations

Relative to the baseline, the best configuration cuts RMSE from 197.1k to 154.1k (a 22% reduction) and raises Adjusted R^2 from 0.667 to 0.794. Figure 3 ranks the top-7 (model, variant) pairs against the baseline, among single models (left) and including the stacking ensembles (right).

5.3 Discussion

Encoding helps, but its effect is model-dependent. Coordinate and target encoding both beat naive categorical encoding across the board, confirming that the location fields carry real, recoverable signal. Which of the two wins, however, depends on the model and is not decisive: for RF the `coord_only` and `tgt` variants are within noise of each other, and combining both (`tgt`) is not uniformly best. We attribute the ambiguity to geography—house values respond to nearby oceans, mountains, and roads that neither a smoothed category mean nor a raw coordinate fully captures.

Stacking recovers signal beyond size. The price / price-per-sqft ensemble improves the stronger models because dividing by living area regularizes the size-dominated, outlier-heavy price target. The best ensemble (`rf/coord_only`, 0.794) beats both the naive baseline (0.667) and the best standard single boosting model (mGBDT/XGBoost on `tgt`, 0.765), supporting the hypothesis that deeper relationships emerge once domain-aware preprocessing offsets the dominance of `sqft_living`.

mGBDT did not justify its depth. A two-layer mGBDT was outperformed by the single-layer (plain XGBoost) configuration: the first XGBoost layer already had sufficient expressive power, and the second layer fit noise rather than additional structure. Consistent with this, the single-layer “mGBDT (XGBoost)” is the best *single* model but does not dominate the tuned ensembles. We therefore adopt XGBoost as the final boosting model and conclude that this dataset lacks the hierarchical complexity that mGBDT is designed to exploit.

6 Conclusion, Limitations, and Future Work

We compared four regression models (MLP, DT, RF, mGBDT) on the *USA House Prices* dataset across four location-encoding variants, both as single models and as a price / price-per-sqft stacking ensemble, all under shared 5-fold cross-validation.

Key findings. (i) Coordinate and target encoding both beat naive categorical encoding for every model, but their relative ranking is model-dependent and statistically inconclusive, reflecting the irregular geography of housing demand. (ii) Price-per-sqft stacking yields a further, consistent improvement for the stronger models by decoupling price from living area and damping outliers. (iii) The best overall configuration, an RF ensemble on coordinate-only features, reaches a validation Adjusted R^2 of 0.794 and cuts RMSE by 22% relative to the `rf/cat` baseline. (iv) A multi-layer mGBDT did not help: its second layer fit noise, so a single target-propagation layer (equivalent to plain XGBoost) was both simpler and stronger, which we adopt as the final boosting model.

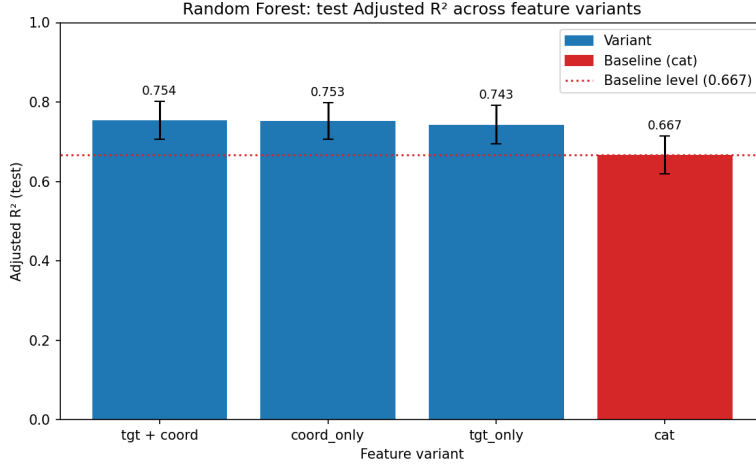


Figure 2: Random Forest validation Adjusted R^2 across the four encoding variants. Every encoded variant beats the naive `cat` baseline (dotted line).

Table 5: Full validation metrics for headline configurations (**price target**). RMSE/MAE in thousand USD; R^2 and Adj. R^2 are across-fold means.

Configuration		RMSE	MAE	MAPE (%)	R^2	Adj. R^2
<code>rf</code> / <code>cat</code>	(baseline)	197.1	112.1	22.85	0.673	0.667
<code>rf</code> / <code>tgt</code>	(single)	168.4	88.5	17.67	0.759	0.754
mGBDT(XGBoost) / <code>tgt</code>	(best single)	164.9	90.4	18.58	0.770	0.765
ensemble <code>rf</code> / <code>tgt</code>	(stacked)	158.0	84.8	17.24	0.787	0.782
ensemble <code>rf</code> / <code>coord_only</code>	(best)	154.1	84.2	17.38	0.798	0.794

Practical implication. The deeper, beyond-size relationships hypothesized at the outset did not emerge merely from a more expressive model; they surfaced only after domain-aware preprocessing (coordinate encoding and price-per-sqft stacking). Feature engineering, paired with a well-regularized tree ensemble, was more decisive than model depth on this tabular task.

Limitations. Our conclusions rest on a single dataset. The two techniques that drove the gains are also dataset-specific: coordinate encoding presumes meaningful latitude/longitude fields, and price-per-sqft stacking presumes a size-dominated, outlier-heavy target. Both may not generalize to other markets or feature sets.

Future work. Three directions follow naturally. First, validate coordinate encoding and price-per-sqft stacking on additional datasets and housing markets to test how far the gains transfer beyond Washington State. Second, instrument the pipeline to record the secondary efficiency metrics (training time, inference time, peak memory), which the current implementation does not yet measure. Third, revisit deeper mGBDT stacks with stronger regularization or richer inverse mappings, to probe whether the layered representation can help on tasks with more hierarchical structure than this one. In future opportunities, we aim to apply mGBDT to complex datasets such as stock data, where more hierarchical learning may be possible.

references

- [1] J. Feng, Y. Yu, and Z.-H. Zhou, “Multi-Layered Gradient Boosting Decision Trees,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, pp. 3551–3561, 2018.
- [2] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [3] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

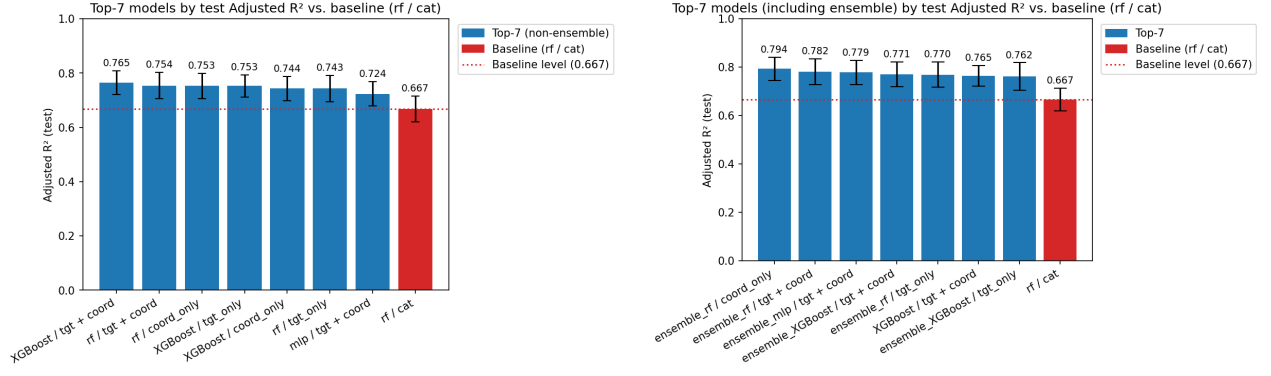


Figure 3: Top-7 configurations by validation Adjusted R^2 vs. the **rf**/**cat** baseline: single models (left) and including stacking ensembles (right). The best, ensemble **rf**/**coord_only**, reaches 0.794.

- [4] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proc. 22nd ACM SIGKDD*, pp. 785–794, 2016.
- [5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009.
- [6] J. H. Friedman, “Stochastic Gradient Boosting,” *Computational Statistics & Data Analysis*, vol. 38, no. 4, pp. 367–378, 2002.